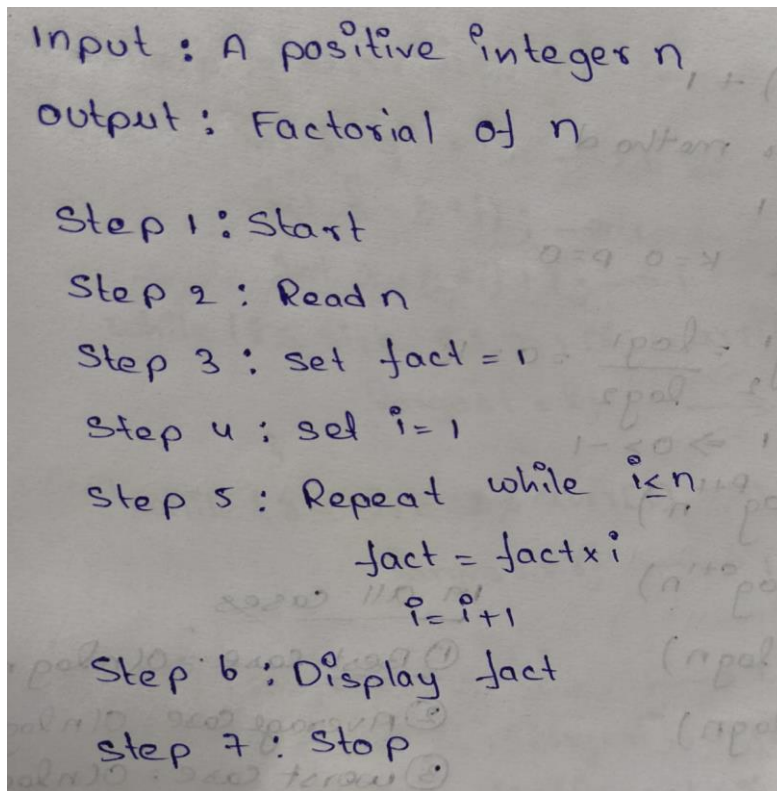


Practical- 4

Aim: Implementation and Time analysis of factorial program using iterative and recursive method.

Iterative Method

Algorithm:



Input : A positive integer n
Output : Factorial of n

Step 1 : Start
Step 2 : Read n
Step 3 : set $fact = 1$
Step 4 : set $i = 1$
Step 5 : Repeat while $i \leq n$
 $fact = fact \times i$
 $i = i + 1$
Step 6 : Display $fact$
Step 7 : Stop

Program:

```
#include <iostream>
using namespace std;
class Factorial
{
public:
    int fact(int n)
    {
        int result = 1;

        for (int i = 1; i <= n; i++)
        {
            result = result * i;
        }

        return result;
    }
};

int main()
{
    Name: Chinni Thabitha
    Enrollment Number: 92460118038
```



```
{
    int n,result;
    Factorial obj;

    cout << "Enter the value to find its factorial: ";
    cin >> n;

    result = obj.fact(n);

    cout << "Factorial of " << n << " is " << result << endl;

    return 0;
}
```

Output:

```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\practical-4\factorial.exe"
Enter the value to find its factorial: 6
Factorial of 6 is 720
```

Time Complexity:

for (i = 1; i <= n; i++) {
 fact = fact * i;
}

$i = 1 \rightarrow 1$
 $i <= n \rightarrow n+1$
 $i++ \rightarrow n$
 $fact = fact * i \rightarrow n$

$T(n) = 1 + (n+1) + n + n$
 $= 3n + 2$
Average $T(n) = O(n)$ $\rightarrow$ worst case

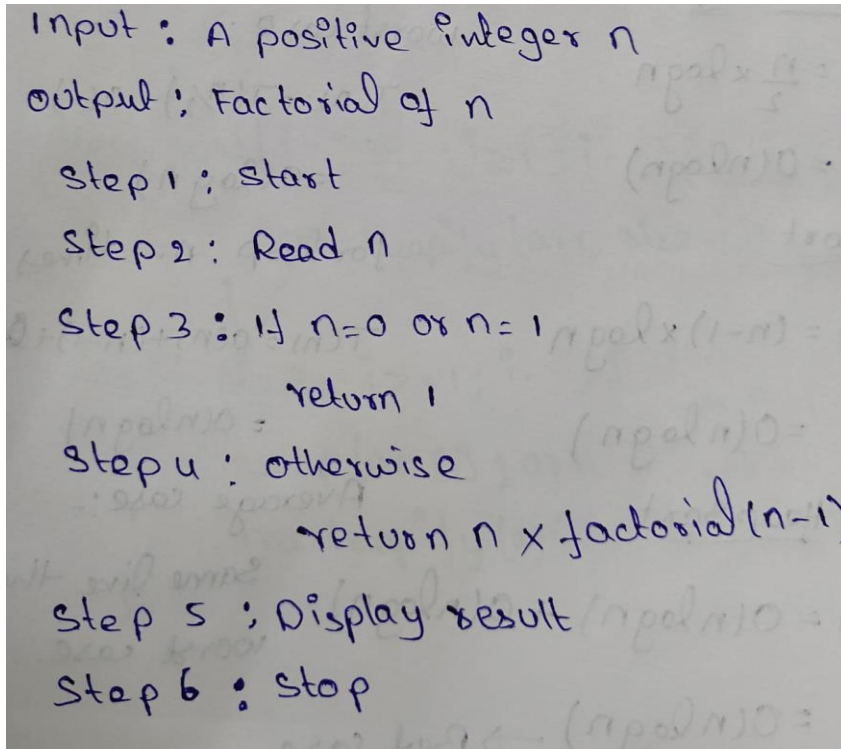
Best Case:-
 $i = 1 \rightarrow 1$
 $i <= n \rightarrow n/2 + 1$
 $i++ \rightarrow n/2$
 $fact = fact * i \rightarrow n/2$

$T(n) = 1 + (n/2 + 1) + n/2 + n/2$
 $= 3n/2 + 2$
 $= O(n)$

Best case
if $n = 1$
 $T(n) = 1 + 2 + 1 + 1 + 1$
 $= 5$
 $= O(1)$

Recursive Method

Algorithm:



Input : A positive integer n
output : Factorial of n
Step 1 : start
Step 2 : Read n
Step 3 : If $n=0$ or $n=1$
 return 1
Step 4 : otherwise
 return $n \times \text{factorial}(n-1)$
Step 5 : Display result
Step 6 : stop

Program:

```
#include <iostream>
using namespace std;

class Factorial
{
public:
    int factr(int n)
    {
        if (n == 0 || n == 1)
        {
            return 1;
        }
        else
        {
            int result = n * factr(n - 1);
            return result;
        }
    }
};

int main()
{
    int n, result;
    Factorial obj;

    Name: Chinni Thabitha
    Enrollment Number:92460118038
```

```
cout << "Enter the value to find its factorial: ";
cin >> n;

result = obj.factr(n);

cout << "Factorial of " << n << " is " << result << endl;

return 0;
}
```

Output:

```
PS C:\mingw\bin> & "C:\Users\chinn\Downloads\dsa in c++\practical-4\recursive.exe"
Enter the value to find its factorial: 6
Factorial of 6 is 720
```

Time complexity:

factorial(n)

if (n == 0 || n == 1) $\rightarrow 1$
return n; $\rightarrow 1$
return n * factorial(n-1); $\rightarrow T(n-1)$

3

$T(n) = T(n-1) + 2$
 $T(n) = T(n-2) + 2 + 2$
 $T(n) = T(n-3) + 2 + 2 + 2$
 $\vdots$
 $T(1) + 2n$
 $= O(n) \rightarrow$ worst case

Best case
when $n = 0$ or $n = 1$:
 $T(n) = 1$
 $= O(1)$

Average case
 $T(n) = T(n-1) + 2$
 $= O(n)$

Conclusion:

The factorial program was implemented using both iterative and recursive methods. The iterative method uses a loop, while the recursive method uses function calls. Both methods have $O(n)$ time complexity in the general case. The iterative method uses $O(1)$ space, whereas the recursive method uses $O(n)$ space due to the recursive call stack.